Resultados de aprendizaje
RAA1 - Aplicar expresiones regulares y teoría de autómatas en la solución de problemas de procesamiento de lenguaje y reconocimiento de patrones.
RAA2 - Aplicar conceptos de gramáticas generativas en la implementación de sistemas de procesamiento de lenguajes especializados para dominios o aplicaciones específicas.
RAA3 - Simplificar gramáticas mediante formas normales para el procesamiento y análisis eficiente de lenguajes.
RAA6 - Comunicar con vocabulario y lenguaje especializado las ideas principales sobre los modelos computacionales estudiados y sus aplicaciones.

Nivel de uso de IAG	Descripción
3. Colaboración con IAG	Los estudiantes pueden apoyarse en la IAG para completar tareas o desarrollar entregables asociados a la actividad, aprovechando las capacidades de estas herramientas para mejorar los productos. Asimismo, se espera que los estudiantes lleven un registro de sus interacciones con la IAG y estén en la capacidad de modificar los resultados generados, demostrando comprensión y dominio conceptual. Este registro debe contener tanto el contenido de autoría propia proporcionado a la IAG como los prompts empleados.

ResumeLens: Formal Language-Based Resume Screening

Problem Statement

Recruitment processes often require reviewing large numbers of résumés to determine whether candidates satisfy the qualifications expected for a particular professional profile. Although résumés usually contain similar types of information—such as education, professional experience, technical skills, contact information, and projects—the way this information is written and organized may vary considerably.

The same qualification may appear using different names, abbreviations, or spelling conventions. For example, JavaScript, Javascript, and JS may refer to the same programming language; similarly, Scikit-learn, sklearn, and scikit learn refer to the same machine-learning library. In addition, different professional profiles require different combinations of qualifications.

In this project, you will develop ResumeLens, an application that processes textual résumés and determines whether the qualifications identified in a candidate's résumé satisfy patterns defined for a professional profile.

ResumeLens will use concepts from formal language theory to address different stages of the problem:

- **Regular expressions** to extract relevant information from résumé text.
- **Finite-state transducers** to normalize different textual representations of equivalent qualifications.
- **Finite automata** to recognize qualification patterns associated with a professional profile.
- **Context-free grammars** to define and validate a structured candidate-profile language.

The purpose of ResumeLens is **not to rank candidates or make hiring decisions**. The application evaluates whether the qualifications explicitly identified in a résumé satisfy formally defined qualification patterns.
The system must support four professional profiles related to software engineering and AI/data-oriented professions. Two profiles are predefined for all teams:

1. **Full Stack Developer**
2. **Machine Learning Engineer**

Each team will additionally define two other professional profiles: one related to software engineering and one related to AI/data. Detailed requirements for these profiles will be provided separately.
The four profiles must be processed through the same general software solution rather than through independent implementations.

Reference Profile 1 — Full Stack Developer

A **Full Stack Developer** works with both client-side and server-side components of software applications. This role commonly involves the development of user interfaces, backend services, APIs, database access, and integration among different software components. For the purposes of this project, a Full Stack Developer profile may include qualifications such as:

- JavaScript or TypeScript
- React, Angular, or Vue
- Node.js, Django, Spring Boot, or a similar backend technology
- SQL or NoSQL databases
- REST APIs
- Git

Then, an example of a résumé fragment could be the following:

> WEDNESDAY ADDAMS
> 3 years of experience developing web applications.
> **Technical Skills**:
> JS, React.js, NodeJS, Postgres, Git.

ResumeLens must process this information through the different stages of the pipeline before determining whether the candidate satisfies an accepted Full Stack Developer qualification pattern.

Reference Profile 2 — Machine Learning Engineer

A **Machine Learning Engineer** combines software development, data processing, and machine-learning techniques to build computational systems that use predictive or learning-based models. For the purposes of this project, a Machine Learning Engineer profile may include qualifications such as:

- Python
- Pandas or NumPy
- Scikit-learn
- TensorFlow, or PyTorch
- Machine-learning model development
- SQL
- Git

Then, an example of a résumé fragment could be the following:

> **Mary Jane Watson**
> 2 years of experience developing predictive models and data-processing pipelines.
> **Technical Skills:**
> Python, Pandas, NumPy, Scikit-learn, TensorFlow, SQL, Git.

As with the Full Stack Developer case, ResumeLens must process the résumé through the complete pipeline before evaluating the qualification pattern.

Departamento de CSI
Computación y Estructuras Discretas III
2026-2 Integrative Task 1

Main Goal

Develop ResumeLens, a résumé-screening system that applies formal language models to extract relevant textual information, normalize equivalent representations, recognize qualification patterns associated with professional profiles, and validate structured candidate information.

The application must support Full Stack Developer, Machine Learning Engineer, and two additional professional profiles defined by each team in the areas of software engineering and AI/data.

Project Goals

1. Resume Information Extraction — Regular Expressions

Objective: Extract relevant textual information from résumés using regular expressions.
Résumés contain information written in different formats. At this stage, the system identifies information that may represent qualifications or candidate data. It does **not** determine whether two expressions are equivalent and does not decide whether the candidate satisfies a job profile.

The first component of ResumeLens must detect elements that may later be used to analyze a candidate, such as:

- Contact information
- Programming languages
- Frameworks and libraries
- Databases
- Academic qualifications
- Professional experience
- Tools and technologies
- Other qualifications relevant to the supported job profiles

Activities for this stage:
For each relevant type of information, you must:

- define the corresponding regular expression;
- explain the language or textual pattern recognized by the expression;
- implement the expression using Python's re module;
- Keep the extracted information in a file or a data structure

Consider the following text in a résumé:

> WEDNESDAY ADDAMS
> 3 years of experience developing web applications.
> **Technical Skills**:
> JS, React.js, NodeJS, Postgres, Git.

The **Stage 1: extraction** may produce the following strings

```
JS
React.js
NodeJS
Postgres
Git
```

⟶ **For the next step:** These strings are the input for the **Stage 2: Normalization**

2. Qualification Normalization — Finite-State Transducers

The same qualification can be expressed in several ways across résumés. ResumeLens must transform those variations into a common representation before the information is analyzed.

Objective

Transform different textual representations of equivalent qualifications into a common canonical representation. This stage addresses variations in abbreviations, naming conventions, and equivalent textual representations.
Finite-state transducers will be used to model these transformations.

Activities for this stage:

For each transducer, you must provide the formal definition (complete 7-tuple) : M = (Q, Σ, Γ, δ, ω, q0, F) and clearly define:

- The set of states Q
- The input alphabet Σ
- The output alphabet Γ
- The transition relation δ
- The output relation ω
- The initial state
- The set of accepting states F
- Provide a graphical representation of the transducers;
- Implement the required transformations using pyformlang.

Processing

Finite-state transducers must be used to model transformations such as:

JS → JAVASCRIPT Javascript → JAVASCRIPT React.js → REACT ReactJS → REACT NodeJS → NODE_JS Node.js → NODE_JS Postgres → POSTGRESQL PostgreSQL → POSTGRESQL	pandas → PANDAS sklearn → SCIKIT_LEARN scikit learn → SCIKIT_LEARN Scikit-learn → SCIKIT_LEARN Tensor Flow → TENSORFLOW TensorFlow → TENSORFLOW Py Torch → PYTORCH PyTorch → PYTORCH

🛑**Note:** These transformations are examples for illustration purposes. You must propose your own transformations

→ Sorting the output of the transformation

The normalized qualifications are prepared as input for the **qualification-pattern recognition stage**.
Before sending them to the classification automata, the application may organize the qualifications in a **canonical order defined by the selected professional profile**. This prevents the result from depending on the order in which the candidate wrote the information in the résumé.
For example, the Full Stack profile may establish the order:

Full Stack Developer
→ Frontend
→ Backend

→ Database
→ Version control

Thus, a résumé containing:
Git, NodeJS, JS, Postgres, React.js

Produces the normalized and sorted sequence:
JAVASCRIPT, REACT, NODE_JS, POSTGRESQL, GIT

3. Qualification Pattern Recognition — Finite Automata

In this stage ResumeLens classify a résumé between the four selected profiles. Then you must determine four profile patterns and use automata to define whether an accepted qualification pattern is satisfied or not.

Objective
Recognize whether the normalized qualifications satisfy an accepted profile pattern for a professional profile.

At this stage **ResumeLens** uses Finite State Automata to classify the résumé according to the four profiles (Full Stack Developer, Machine Learning Engineer and the two selected by the students). For this purpose, students may use:

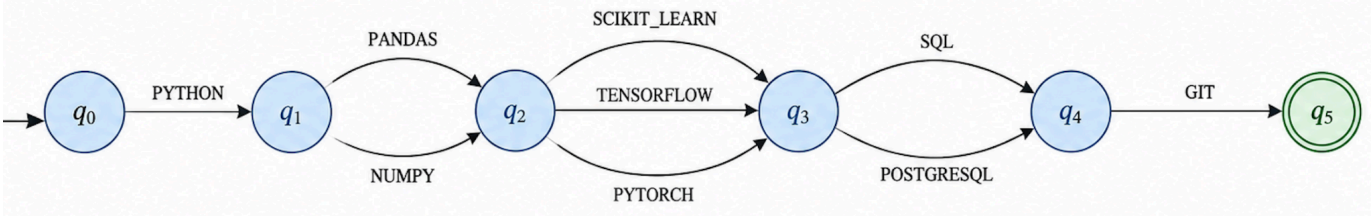
- Deterministic finite automata;
- Nondeterministic finite automata;
- Nondeterministic finite automata with ε/λ-transitions;

Activities for this stage:
For each automaton, you must provide the formal definition (complete 5-tuple) : M = (Q, Σ, δ, q0, F) and clearly define:

- The set of states Q
- The alphabet Σ
- The transition function/relation δ
- The initial state
- The set of accepting states F
- Explain: What is the type of automaton (DFA, NFA, or ε-NFA) support your answer considering the previous definition
- Provide its transition diagram
- Explain the profile pattern represented by the automaton
- Implement the automaton using pyformlang.

Example

For the sequence (from previous stage): PYTHON, PANDAS, TENSORFLOW, POSTGRESQL, GIT



Profile patter: MACHINE_LEARNING_ENGINEER
Output: ACCEPTED

4. Candidate Profile Language — Context-Free Grammars using textX

After the information extraction, normalization and classification, **ResumeLens** produces a Résumé specification and visualization using Domain Specific Languages and the TextX library.

Objective
Define a domain-specific language for representing the structured information produced by ResumeLens. The purpose of this component is different from extraction, normalization, and qualification recognition. The grammar defines how the resulting candidate information must be structurally represented.

Input
The input in this stage is the structured information obtained during the previous stages, such as:

- Candidate information
- Normalized qualifications
- Result of the profile classification

Processing
You must define a DSL for the candidate profile specification using a context-free grammar and implemented using textX. The language should allow information such as: personal information, contact information, experience, skills, and the result of the profile classification (for each accepted profile). The language should support structured and repeated elements where appropriate, such as multiple professional experiences, education records, or skills.

Activities for this stage:
For your DSL definition you must

- Define the grammar using EBNF including the identification of terminals and non-terminals;
- Explain the structural characteristics of the language;
- Implement the grammar using textX;
- Validate generated candidate profiles;
- Reject representations that violate the lexical or syntactic rules.
- Generate the visualization output

Visualization Output
Once a candidate profile has been successfully validated, the application must use its structured information to generate a short **HTML or Markdown visualization** of the candidate.

For example, a valid candidate profile might produce the following HTML code [Link] download the code and open it using a browser.

Literature Review:
Before implementing your program, you should conduct a review of the literature associated with the problem at hand. This is to guide the project, avoid redundancies, make informed decisions, develop a solid theoretical foundation, and thus obtain results of high impact and relevance.

Deliverables

1. Research poster: [Guide]
2. Design:
    a. Design of modules (functions, inputs-outputs)
    b. Formalization

   c. Design of test cases, including scenarios.
3. Python implementation:
   a. Complete and correct implementation of the model, UI, and tests

Project Presentation

Prepare a 10-minute technical presentation (in English) summarizing your project. The presentation must clearly explain the problem addressed, the formal models applied, the system architecture, and the main results obtained. Use your poster to prepare the presentation.

The presentation must be analytical and technical in nature. It should:

- Clearly define the security problem and its relevance in the context of code repositories.
- Explain the methodology, explicitly connecting each system component to its corresponding formal model (regular expressions, finite automata, finite state transducers, context-free grammars).
- Describe the design decisions made (e.g., abstraction level, alphabet definition, grammar structure, classification criteria).
- Present and interpret results, including examples of detection, classification, transformation, and validation.
- Discuss limitations and possible improvements.

This is not a product demonstration or marketing presentation. The focus must be on modeling choices, theoretical justification, implementation strategy, and the rationale of your decisions as engineers to be. Students are expected to demonstrate conceptual understanding of the formal language models they implemented and their practical implications.

Teams

This project can be done in groups of minimum 2 and maximum 3 people. The team must be made up solely and exclusively of people from the same group (class/section).

You should create your own GitHub repository and then put the link in the File where you register your team. Please, fill the following form to register your team, keep the prefix En (where n is a number) in the name of your team [link]. You also need to fill your Course Code (09772 if you saw this assignment on Intu, 09834 if you saw this on Saman) and the number of group that the members of the team attend. 1 for professor Angela, 3 for professor Andres, 5 for Juan Marcos.

Your repository must have at least 10 commits with a difference of at least 2 hours between each of them. These commits must be meaningful in terms of value (e.g., they cannot be simply deleting a variable or changing a method name). In the repository or project, there should be a directory called docs/ where each of the design documents should be placed, documents in your repository should be written using markdown. Include any necessary clarifications for manipulating or understanding your project in the readme.md file as extra documentation (e.g., IDE used, members' names). The contribution of each member will be evaluated based on their input, and the only way to verify this will be through the commits.

Note: The poster as well as the presentation or demonstration showcasing the functionality and features of the system must be in english.

Deadline for Submission: 11th of October, 2026